

```

1  #include "FibonacciSequence.h"
2
3  // Constructor - Initializes the sequence with by setting
   fPrevious to 0 and fCurrent to 1.
4  FibonacciSequence::FibonacciSequence() noexcept
5      : fPrevious(0), fCurrent(1)
6  {
7  }
8
9  // Operator* - Returns a constant reference to the current
   Fibonacci value.
10 const uint64_t& FibonacciSequence::operator*() const
    noexcept {
11     return fCurrent;
12 }
13
14 // Prefix Increment operator - Advances the Fibonacci
   sequence.
15 FibonacciSequence& FibonacciSequence::operator++()
    noexcept {
16     uint64_t temp = fCurrent;           // Save the
        current value
17     fCurrent = fCurrent + fPrevious;    // Calculate the
        next Fibonacci number
18     fPrevious = temp;                  // Update
        fPrevious to the old current value
19     return *this;                      // Return the
        updated object
20 }
21
22 // Postfix increment operator
23 FibonacciSequence FibonacciSequence::operator++(int)
    noexcept {
24     FibonacciSequence temp = *this;    // Saves the
        current state
25     ++(*this);                         // Advances the
        iterator
26     return temp;                       // Returns the old
        state
27 }
28
29 // Equality operator - Checks if two FibonacciSequence
   objects are equal by comparing their fPrevious and
   fCurrent values.

```

```
30 bool FibonacciSequence::operator==(const FibonacciSequence
    & other) const noexcept {
31     return (fPrevious == other.fPrevious) && (fCurrent ==
        other.fCurrent);
32 }
33
34 // begin - Resets the sequence to its starting values (0
    and 1)
35 void FibonacciSequence::begin() noexcept {
36     fPrevious = 0;
37     fCurrent = 1;
38 }
39
40 // end() - Sets the sequence to a state (fPrevious and
    fCurrent are zero) that does not occur in valid Fibonacci
    generation.
41 void FibonacciSequence::end() noexcept {
42     fPrevious = 0;
43     fCurrent = 0;
44 }
45
```